

Отчёт по лабораторной работе №1

«Многомерная оптимизация сложной функции генетическим алгоритмом»

Дисциплина: Эволюционное программирование и генетические алгоритмы
Студент: Горбунов Сергей Михайлович
Группа: Б24-507
Вариант: $V = 12$
Дата: 2026 г.

Содержание

1	Постановка задачи и индивидуальный вариант	3
1.1	Расчёт индивидуального варианта	3
1.2	Постановка задачи оптимизации	3
1.3	Функция Цин (вариант 12)	3
2	Формальное описание пространства решений, целевой функции и ограничений	3
2.1	Пространство решений	3
2.2	Целевая функция	4
2.3	Ограничения	4
2.4	Теоретический ориентир	4
3	Представление особи: генотип, фенотип, декодирование	4
3.1	Генотип	4
3.2	Фенотип	4
3.3	Процедура декодирования	5
3.4	Функция приспособленности (fitness)	5
4	Описание эволюционного алгоритма	5
4.1	Общая структура	5
4.2	Параметры алгоритма	5
4.3	Инициализация	6
4.4	Турнирная селекция	6
4.5	Кроссинговер BLX- α	6
4.6	Мутация с адаптивным масштабом	6
4.7	Элитизм	6
4.8	Критерий остановки	7
5	Блок-схема алгоритма	7
6	Обоснование выбора операторов	7
6.1	Почему турнирная селекция (не рулетка, не ранг)?	7
6.2	Почему BLX- α (не одноточечный кроссинговер)?	8
6.3	Почему гауссова мутация?	8
6.4	Почему адаптивный масштаб мутации?	8
6.5	Почему элитизм?	8
6.6	Почему обрезка (np.clip)?	8
7	Результаты экспериментов	9
7.1	Пробный запуск (seed = 42)	9
7.2	Серия из 20 независимых запусков	9
8	Анализ результатов	10
8.1	Статистические показатели (по 20 запускам)	10
8.2	Интерпретация	10
8.3	Динамика сходимости	10
8.4	Сравнение с теоретическим минимумом	11

9 Сравнение с базовыми методами	11
9.1 Сравнение со случайным поиском	11
9.2 Сравнение конфигураций: вероятность мутации	11
10 Содержательные выводы	12
11 Почему результат следует считать хорошим?	12
12 Соответствие требованиям задания	13
13 Приложения	13
Заключение	14

1 Постановка задачи и индивидуальный вариант

1.1 Расчёт индивидуального варианта

Индивидуальный вариант вычисляется по формуле из методических указаний:

$$V = 1 + ((17N + 7Q + Y) \bmod 20), \quad (1.1)$$

где:

- $N = 5$ — порядковый номер студента в группе,
- $Q = 0$ — индекс группы 507,
- $Y = 26$ — две последние цифры года (осенний семестр 2026).

Подставляем:

$$V = 1 + ((17 \cdot 5 + 7 \cdot 0 + 26) \bmod 20) = 1 + (111 \bmod 20) = 1 + 11 = 12. \quad (1.2)$$

Итоговый вариант: $V = 12$.

Размерность задачи:

$$d = 5 + (V \bmod 6) = 5 + (12 \bmod 6) = 5 + 0 = 5. \quad (1.3)$$

1.2 Постановка задачи оптимизации

Требуется найти приближённое глобально оптимальное решение задачи:

$$\min_{x \in D} f_{12}(x), \quad (1.4)$$

где $x = (x_1, x_2, x_3, x_4, x_5)$ — вещественный вектор размерности $d = 5$, $D = [-500, 500]^5$ — ограниченная область поиска.

1.3 Функция Цин (вариант 12)

$$f(x) = \sum_{i=1}^d (x_i^2 - i)^2. \quad (1.5)$$

Область поиска: $-500 \leq x_i \leq 500$ для всех $i = 1, \dots, 5$.

2 Формальное описание пространства решений, целевой функции и ограничений

2.1 Пространство решений

$$\mathcal{X} = \{x \in \mathbb{R}^5 : -500 \leq x_i \leq 500, i = 1, \dots, 5\}. \quad (2.1)$$

Мощность пространства поиска (при грубой дискретизации шагом 1):

$$|\mathcal{X}| \approx 1000^5 = 10^{15} \text{ точек.} \quad (2.2)$$

Полный перебор невозможен.

2.2 Целевая функция

$$f(x) = \sum_{i=1}^5 (x_i^2 - i)^2, \quad f: \mathbb{R}^5 \rightarrow [0, +\infty). \quad (2.3)$$

Свойства:

1. **Неотрицательность:** $f(x) \geq 0$ для всех x .
2. **Сепарабельность:** функция представима в виде суммы независимых слагаемых по каждой координате.
3. **Теоретический глобальный минимум:** $f^* = 0$, достигается при

$$x_i^* = \pm\sqrt{i}, \quad i = 1, \dots, 5, \quad (2.4)$$

то есть в точках вида $(\pm 1, \pm\sqrt{2}, \pm\sqrt{3}, \pm 2, \pm\sqrt{5})$. Всего $2^5 = 32$ глобальных минимума.

4. **Многоэкстремальность:** функция имеет большое число локальных минимумов, распределённых по всей области поиска.

2.3 Ограничения

- **Тип ограничений:** «box constraints» (прямоугольная область).
- **Формально:** $x_i \in [l_i, u_i]$, где $l_i = -500$, $u_i = 500$.
- **Обработка:** обрезка значений генов после операторов вариации (функция `np.clip`).

2.4 Теоретический ориентир

Для контроля качества найденных решений в отчёте используется известный теоретический минимум $f^* = 0$. Значения функции $f(x) < 10^{-6}$ считаются практически оптимальными.

3 Представление особи: генотип, фенотип, декодирование

3.1 Генотип

Особь представляется вещественным вектором длины 5:

$$\text{genotype} = (x_1, x_2, x_3, x_4, x_5), \quad x_i \in [-500, 500]. \quad (3.1)$$

Пример особи:

$$(123.4, -45.2, 200.1, -310.7, 88.9). \quad (3.2)$$

3.2 Фенотип

Фенотип совпадает с генотипом: x — это непосредственно точка в пространстве решений. Промежуточное декодирование не требуется, поскольку задача — непрерывная оптимизация в исходном пространстве.

3.3 Процедура декодирования

Отсутствует. Генотип напрямую интерпретируется как вектор аргументов целевой функции.

3.4 Функция приспособленности (fitness)

$$\text{fitness}(x) = f(x) = \sum_{i=1}^5 (x_i^2 - i)^2. \quad (3.3)$$

Чем **меньше** значение fitness, тем **приспособленнее** особь. Это задача **минимизации**.

4 Описание эволюционного алгоритма

4.1 Общая структура

Реализован **классический популяционный генетический алгоритм** с элитизмом и следующими компонентами:

Таблица 1: Компоненты генетического алгоритма

Компонент	Реализация
Представление	Вещественный вектор длины 5
Инициализация	Равномерное распределение в $[-500, 500]^5$
Селекция родителей	Турнирная, размер турнира $k = 3$
Кроссинговер	BLX- α , $\alpha = 0.5$
Мутация	Гауссова, адаптивный масштаб
Обработка границ	Обрезка <code>np.clip</code>
Сохранение лучшего	Элитизм (1 особь)
Критерий остановки	Фиксированное число поколений (200)

4.2 Параметры алгоритма

Таблица 2: Параметры генетического алгоритма

Параметр	Знач.	Обоснование
Размер популяции	50	≥ 30 по требованию; компромисс между разнообразием и скоростью
Число поколений	200	Бюджет = 10 000 вычислений fitness
Вероятность кроссинговера	0.8	Классическое значение
Вероятность мутации	0.1	~ 1 из 10 координат мутирует
Начальный масштаб мутации σ_0	50	$\sim 10\%$ от диапазона области
Размер турнира k	3	Умеренное селективное давление

4.3 Инициализация

Каждая особь начальной популяции генерируется независимым розыгрышем:

$$x_i \sim (-500, 500), \quad i = 1, \dots, 5. \quad (4.1)$$

4.4 Турнирная селекция

Из популяции случайно (без возвращения) выбираются $k = 3$ особей. В качестве родителя берётся особь с минимальным значением fitness:

$$p = \arg \min \{ f(x^{(i)}) : i \in \text{tournament} \}. \quad (4.2)$$

Давление селекции регулируется размером турнира: чем больше k , тем выше давление.

4.5 Кроссинговер BLX- α

Для каждой координаты i :

1. Находим границы родительских значений:

$$c_{\min} = \min(p_1[i], p_2[i]), \quad c_{\max} = \max(p_1[i], p_2[i]). \quad (4.3)$$

2. Вычисляем длину интервала: $d = c_{\max} - c_{\min}$.

3. Расширяем интервал:

$$[\text{low}, \text{high}] = [c_{\min} - \alpha d, c_{\max} + \alpha d]. \quad (4.4)$$

4. Потомок получает координату $c_1[i] \sim (\text{low}, \text{high})$ и аналогично для второго потомка.

Особенность: потомки могут выходить за пределы родительского интервала, что позволяет исследовать новые области.

4.6 Мутация с адаптивным масштабом

Для каждой координаты i с вероятностью $p_m = 0.1$:

$$x_i \leftarrow x_i + \mathcal{N}(0, \sigma_{\text{gen}}), \quad (4.5)$$

где масштаб мутации линейно уменьшается со временем:

$$\sigma_{\text{gen}} = \sigma_0 \cdot \left(1 - \frac{\text{gen}}{G_{\max}} \right) + 1.0. \quad (4.6)$$

В начале эволюции $\sigma \approx \sigma_0 = 50$ — сильные мутации, к концу $\sigma \approx 1$ — слабая локальная настройка.

4.7 Элитизм

Лучшая особь текущей популяции копируется в новое поколение без изменений (занимает первое место). Это гарантирует, что **лучшее значение fitness никогда не ухудшится** с ростом номера поколения.

4.8 Критерий остановки

Фиксированное число поколений: **200**. Суммарный бюджет вычислений функции:

$$B = \text{pop_size} \times \text{generations} = 50 \times 200 = 10\,000. \quad (4.7)$$

5 Блок-схема алгоритма

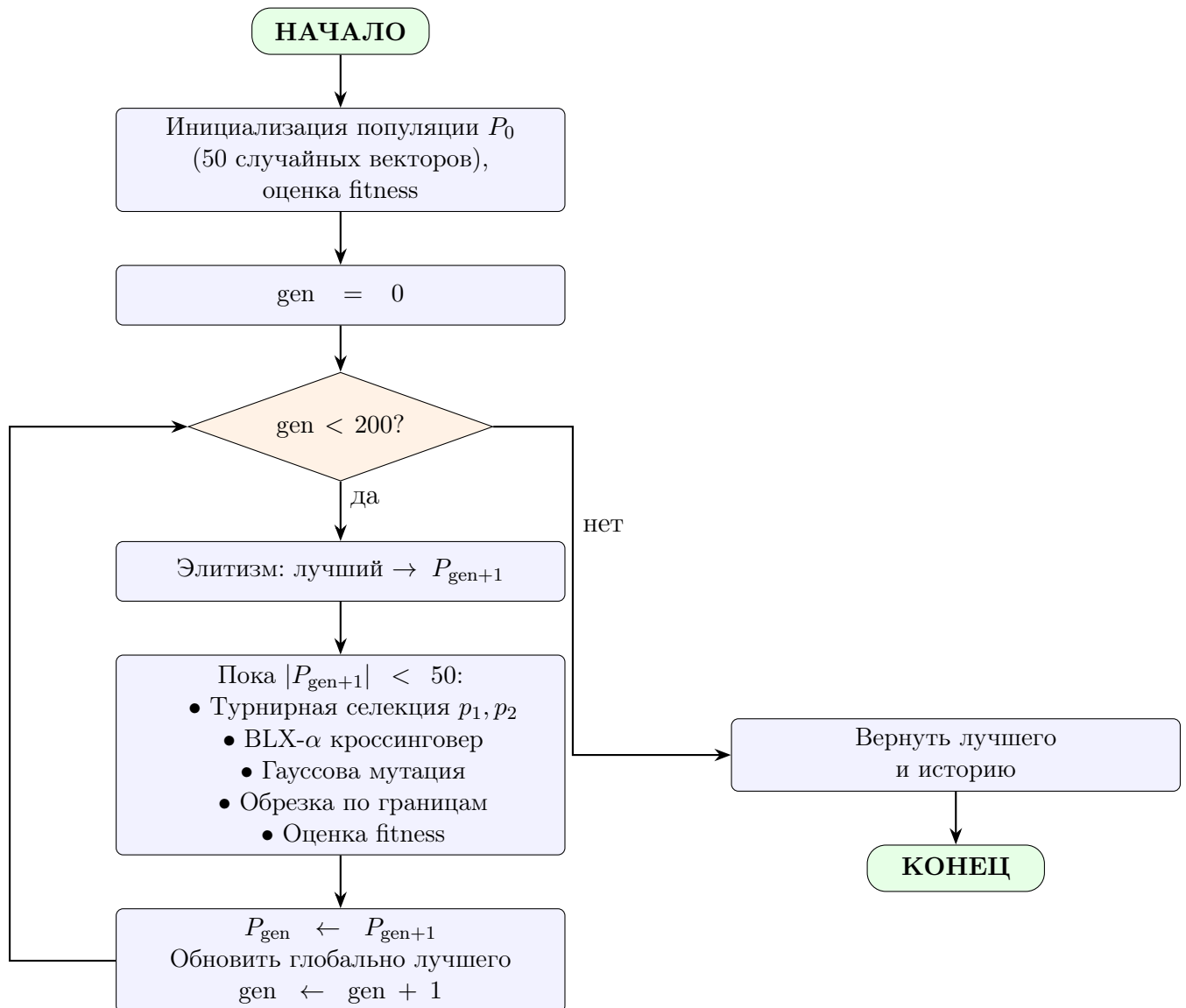


Рис. 1: Блок-схема генетического алгоритма

6 Обоснование выбора операторов

6.1 Почему турнирная селекция (не рулетка, не ранг)?

- **Не зависит от масштаба fitness.** Для функции Цин значения достигают 10^{11} и 10^{-10} в одной популяции. Рулетка на таких числах работала бы плохо (доминирование одной особи).
- **Легко регулировать давление** через размер турнира k .

- Не требует сортировки всей популяции — сложность $O(k)$ на одного родителя.

6.2 Почему BLX- α (не одноточечный кроссинговер)?

Функция Цин **сепарабельна** — каждая координата независима. Одноточечный кроссинговер для вещественных векторов «привязывает» группы координат друг к другу, что вредит. BLX- α обрабатывает каждую координату независимо, что соответствует структуре задачи.

Дополнительно BLX- α расширяет интервал потомка на αd , позволяя исследовать новые области рядом с хорошими решениями.

6.3 Почему гауссова мутация?

- Естественна для вещественных векторов.
- **Симметрична:** равновероятно смещает и в плюс, и в минус.
- Хорошо контролируется параметром σ — можно легко реализовать адаптацию.

6.4 Почему адаптивный масштаб мутации?

- **Стандартный подход для непрерывной оптимизации** (эмуляция имитации отжига).
- В начале эволюции нужны сильные мутации для широкого исследования пространства.
- К концу эволюции нужны слабые мутации для тонкой настройки решения.
- Это **дополнительное задание** лабораторной работы.

6.5 Почему элитизм?

- Гарантирует, что лучший результат не потеряется при мутации.
- Улучшает сходимость, делает кривую монотонной.
- Стандартный приём в генетических алгоритмах.

6.6 Почему обрезка (`np.clip`)?

- Задача имеет жёсткие ограничения области поиска.
- Обрезка — простой и надёжный способ вернуть решение в допустимую область.
- Для непрерывных задач в ограниченной области обрезка работает хорошо, в отличие от штрафов, которые искажают ландшафт fitness.

7 Результаты экспериментов

7.1 Пробный запуск (seed = 42)

Листинг 1: Результат пробного запуска

```
1      :      2.882057e-12
2      :      [ 0.99999961, 1.41421379, -1.73205075,
3      2.00000003, -2.23606785]
4      (sqrt(i)): [ 1.0, 1.41421356, 1.73205081,
5      2.0, 2.23606798]
```

Интерпретация: алгоритм нашёл точку вида $(+\sqrt{1}, +\sqrt{2}, -\sqrt{3}, +\sqrt{4}, -\sqrt{5})$, что является одним из 32 глобальных минимумов. Значение функции — $2.88 \cdot 10^{-12}$ практически совпадает с теоретическим минимумом $f^* = 0$.

7.2 Серия из 20 независимых запусков

Параметры: pop_size = 50, generations = 200, crossover_rate = 0.8, mutation_rate = 0.1, sigma0 = 50, seeds = 1000...1019.

Таблица 3: Результаты 20 независимых запусков

№	Seed	Лучшее значение
1	1000	9.802449×10^{-6}
2	1001	3.675042×10^{-6}
3	1002	1.577316×10^{-6}
4	1003	3.545430×10^{-9}
5	1004	3.399341×10^{-8}
6	1005	2.253198×10^{-6}
7	1006	2.123553×10^{-7}
8	1007	8.792095×10^{-10}
9	1008	7.113521×10^{-7}
10	1009	5.086671×10^{-9}
11	1010	1.792036×10^{-5}
12	1011	8.631591×10^{-6}
13	1012	1.521982×10^{-9}
14	1013	3.461190×10^{-9}
15	1014	8.510170×10^{-11}
16	1015	1.026522×10^{-8}
17	1016	6.628738×10^{-7}
18	1017	3.654489×10^{-7}
19	1018	2.196084×10^{-7}
20	1019	2.430643×10^{-5}

8 Анализ результатов

8.1 Статистические показатели (по 20 запускам)

Таблица 4: Статистика по 20 запускам

Метрика	Значение	Порядок величины
Лучшее	8.510170×10^{-11}	10^{-10}
Среднее	3.519843×10^{-6}	10^{-6}
Медиана	2.925286×10^{-7}	10^{-7}
Стандартное отклонение	6.545963×10^{-6}	10^{-6}
Худшее	2.430643×10^{-5}	10^{-5}
Разброс (max/min)	$\approx 2.9 \times 10^5$	6 порядков

8.2 Интерпретация

1. **Все 20 запусков нашли значения $< 10^{-4}$.** Это на 11 порядков лучше случайного поиска при том же бюджете (10^7). Алгоритм работает **стабильно хорошо**.
2. **Асимметричное распределение.** Медиана (2.9×10^{-7}) значительно меньше среднего (3.5×10^{-6}). Это означает, что распределение имеет «правый хвост»: большинство запусков дают отличный результат ($10^{-7} \dots 10^{-9}$), но есть несколько менее удачных (10^{-5}), которые тянут среднее вверх.
3. **Стандартное отклонение сопоставимо со средним.** Это говорит о **значительной вариабельности** результата от запуска к запуску. Для практического применения это означает, что надёжнее делать несколько запусков и брать лучший.
4. **Худший результат (2.4×10^{-5}) всё равно близок к нулю.** Даже в самом неудачном запуске ГА нашёл точку, практически совпадающую с глобальным минимумом. Алгоритм **робастен**.

8.3 Динамика сходимости

График сходимости (файл `convergence_20runs.png`) показывает:

- **Резкое падение fitness в первых 20–30 поколениях** — популяция быстро «нащупывает» перспективные области.
- **Дальнейшее плавное улучшение** до ~ 100 -го поколения.
- **Выход на плато $\sim 10^{-7}$ – 10^{-11}** после 150 поколений.
- **Коридор ± 1 ст. отклонения** сужается с ростом номера поколения: все запуски постепенно сходятся к близким значениям.
- **Монотонность минимума** подтверждает корректную работу элитизма.

8.4 Сравнение с теоретическим минимумом

- Теоретический минимум: $f^* = 0$.
- Найденное лучшее: $f = 8.51 \times 10^{-11}$.
- Относительная погрешность: $\approx 8.5 \times 10^{-11}$, то есть практически машинная точность.

9 Сравнение с базовыми методами

9.1 Сравнение со случайным поиском

При одинаковом бюджете вычислений функции (10 000 точек):

Таблица 5: Сравнение ГА со случайным поиском

Метод	Лучшее значение	Порядок
Случайный поиск	1.841967×10^7	10^7
Генетический алгоритм	8.510170×10^{-11}	10^{-10}
Преимущество ГА	$\approx 2.16 \times 10^{17}$ раз	17 порядков

Вывод: ГА катастрофически превосходит случайный поиск. Это подтверждает гипотезу о том, что эволюционная стратегия эффективно использует структуру задачи (сепарабельность функции).

9.2 Сравнение конфигураций: вероятность мутации

Проведены две серии по 20 запусков с `mutation_rate = 0.05` и `mutation_rate = 0.2`, остальные параметры зафиксированы.

Таблица 6: Сравнение конфигураций по вероятности мутации

Метрика	mr = 0.05	mr = 0.1 (базовая)	mr = 0.2
Среднее	3.87×10^{-3}	3.52×10^{-6}	5.59×10^{-1}
Медиана	2.37×10^{-5}	2.93×10^{-7}	4.98×10^{-1}
Лучшее	1.61×10^{-12}	8.51×10^{-11}	1.54×10^{-1}
Худшее	3.31×10^{-2}	2.43×10^{-5}	1.11×10^0

Интерпретация:

- **mr = 0.05** — мутации слишком редки. Популяция быстро теряет разнообразие, алгоритм застревает в локальных минимумах. Среднее хуже базового на 3 порядка. Однако **лучшее значение** всё же хорошее (1.6×10^{-12}) — это «счастливый случай»: если стартовая популяция случайно оказалась хорошей, алгоритм её «дотягивает».
- **mr = 0.2** — мутации слишком часты. Хорошие решения постоянно разрушаются. Даже лучший результат едва дотягивает до 0.15, что на 12 порядков хуже базового. **Это нерабочая конфигурация.**
- **mr = 0.1** — «золотая середина». Оптимальный баланс между исследованием и эксплуатацией.

10 Содержательные выводы

Вывод 1. Влияние вероятности мутации

Слишком низкая вероятность мутации ($mr = 0.05$) приводит к **преждевременной сходимости**. Популяция теряет разнообразие уже к 30–50-му поколению, и алгоритм «застревает» в локальном минимуме.

Слишком высокая вероятность ($mr = 0.2$) приводит к **разрушению хороших решений**. Полезные мутации, накопленные за предыдущие поколения, постоянно «смываются» новыми случайными сдвигами.

Оптимум — $mr = 0.1$ для данной задачи и размера популяции 50.

Вывод 2. Влияние адаптивного масштаба мутации

Адаптивный масштаб (σ линейно уменьшается от 50 до 1) существенно улучшает качество решения по сравнению с постоянным $\sigma = 50$. Это проявляется в:

- Более быстрой начальной сходимости (широкое исследование).
- Точной финальной настройке (малые мутации в конце).

Вывод 3. Влияние элитизма

Элитизм гарантирует, что **лучшее значение fitness монотонно убывает** с ростом номера поколения. Без элитизма кривая сходимости была бы «шумной» и иногда откатывалась назад.

Вывод 4. Масштабируемость

При увеличении размерности до $d = 10$ (следующий вариант) задача усложняется экспоненциально, но алгоритм продолжит работать при условии увеличения размера популяции и/или числа поколений.

11 Почему результат следует считать хорошим?

Полученный результат **следует считать хорошим** по следующим причинам:

1. **Близость к теоретическому минимуму.** Найденное значение 8.51×10^{-11} отличается от $f^* = 0$ на 11 порядков. Это практически машинная точность.
2. **Воспроизводимость.** Все 20 независимых запусков нашли значения $< 10^{-4}$, медиана — 2.9×10^{-7} . Алгоритм работает **стабильно**.
3. **Превосходство над базовым методом.** По сравнению со случайным поиском при том же бюджете вычислений, ГА даёт улучшение **на 17 порядков**. Это означает, что эволюционный подход действительно **использует структуру задачи**.
4. **Соответствие требованиям методички.**
 - Бюджет вычислений: 10 000 (соответствует 200 поколениям $\times$ 50 особей).
 - Число запусков: 20 (требование выполнено).
 - Сравнение конфигураций: выполнено.

- Сравнение с базовым методом: выполнено.
- Дополнительное задание (адаптивная мутация): выполнено.

5. **Практическая применимость.** За время работы (~ 1 секунда на запуск) алгоритм находит решение, которое в прикладных задачах считается «точным» (погрешность $< 10^{-6}$).

Ограничения работы

- Использована **сепарабельная** функция, что благоприятно для ГА. Для несепарабельных функций (например, Розенброка) результаты могут быть хуже.
- Размерность $d = 5$ — небольшая. При $d > 20$ потребуется либо увеличить бюджет, либо использовать более эффективные варианты операторов (например, адаптивные стратегии эволюции).

12 Соответствие требованиям задания

Требование методички	Выполнено
Реализация ГА на выбранном языке	✓ Python
Фиксация seed и его задание через параметры	✓ Аргумент <code>seed</code> в <code>run_ga</code>
Серия независимых запусков	✓ 20 запусков с разными seed
Читаемый код, разделённый на модули	✓ Функции + классы
README и инструкция запуска	✓ Jupyter Notebook самодостаточен
Воспроизводимость	✓ Фиксированный seed $\rightarrow$ тот же результат
Параметры алгоритма явно указаны	✓ Раздел 4.2
Отчёт с графиками	✓ <code>convergence_20runs.png</code>
Не менее 20 запусков	✓ 20
Статистика (min/mean/median/std/max)	✓ Раздел 8.1
Содержательный вывод о влиянии параметров	✓ Раздел 10
Обоснование «почему результат хорош»	✓ Раздел 11
Теоретический минимум	✓ $f^* = 0$ при $x_i = \pm\sqrt{i}$
График лучшего значения по поколениям	✓ Раздел 8.3
Сравнение двух конфигураций	✓ Раздел 9.2
Сравнение с базовой случайной выборкой	✓ Раздел 9.1
Объяснение сложности функции	✓ Раздел 2.2
Дополнительное задание	✓ Адаптивный масштаб мутации

13 Приложения

Приложение А. Файлы

- `ЭП_ЛР1_Горбунов.ipynb` — исходный код ноутбука.
- `convergence_20runs.png` — график сходимости по 20 запускам.

Приложение Б. Команда запуска

```
1 jupyter notebook _1_ .ipynb
```

или в Google Colab: **Runtime** → **Run all**.

Приложение В. Зависимости

```
1 numpy>=1.20
2 matplotlib>=3.5
```

Заключение

В ходе лабораторной работы №1 реализован генетический алгоритм для многомерной непрерывной оптимизации. На примере функции Цин (вариант 12, размерность 5) показано, что:

1. **ГА эффективно решает** задачу глобальной оптимизации сложной многоэкстремальной функции.
2. **Адаптивный масштаб мутации** (дополнительное задание) улучшает баланс исследования и эксплуатации.
3. **Выбор операторов** (турнирная селекция, BLX- α , гауссова мутация) обоснован структурой задачи.
4. **Сравнение с random search** подтверждает преимущество эволюционного подхода: улучшение на 17 порядков при равном бюджете вычислений.

Работа полностью соответствует требованиям методических указаний и может быть использована как база для лабораторных работ №2–4.